

HYBRID MUSIC RECOMMENDATION SYSTEM

Sagar Pundir (Roll Number: 23035010935)

Term Project Report
Trimester 7

B.Sc. (Honours) Data Science and Artificial Intelligence
Indian Institute of Technology Guwahati, India
sagar.pundir@op.iitg.ac.in

ABSTRACT

This report details the development of a hybrid music recommendation system to enhance user engagement and minimize churn on digital music platforms. Recommendation systems are essential for personalization, however traditional methods such as popularity or content-based filtering often suffer from bias or the "cold start" problem. To overcome this, I created a content-based filtering (using song metadata) and collaborative filtering (using user listening history). I addressed the challenge of processing a massive dataset—containing nearly 9.7 million users—by using Dask for parallel computation and Scipy's sparse matrices to save memory. A weighted sum approach was used to merge the scores, allowing for a balance between similarity and diversity. The project was managed using DVC and GitHub to ensure reproducibility and efficient version control.

Index Terms— Recommendation System, Content-Based Filtering, Collaborative Filtering, Cosine Similarity, Dask, DVC

1. INTRODUCTION

Music streaming platforms today carry millions of tracks, far more than any listener could explore manually. A recommendation system solves this by automatically surfacing songs a user is likely to enjoy, based on either the properties of the music itself or the habits of users with similar taste.

Without this kind of system, users tend to loop through the same familiar songs and gradually lose interest in the platform. A well-designed recommender keeps engagement high by consistently showing people something worth listening to.

In this project, I built a hybrid recommendation system that combines content-based filtering and collaborative filtering. Using both together allows the system to stay personalised while also introducing enough variety to keep recommendations interesting. The system takes a song as input and returns a ranked list of suggestions using a weighted combination of scores from both modules.

Section 2 defines the problem and objectives. Section 3 covers the methodology. Section 4 presents the experimental setup and results. Section 5 discusses key observations and challenges. Section 6 gives the conclusion and future work. Section 7 contains links to the project artifacts.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1. Problem Statement

The core question this project tries to answer is: given a song a user is currently listening to, how can we recommend other songs they will actually enjoy? Suggesting the most globally popular tracks does not work well here because it gives everyone the same list and ignores individual preferences entirely.

Each standard approach on its own has a known weakness. Content-based filtering tends to over-specialise, meaning the system keeps recommending tracks that are too similar and the user never discovers anything new. Collaborative filtering is harder to scale because the user-item matrix grows very large very quickly. Both approaches also share the cold start problem: a brand new user or a newly added song has no prior data attached to it, so the system has nothing to base a recommendation on.

In this project specifically, the dataset contains around 9.7 million unique users and 30,000 unique songs. Constructing and querying an interaction matrix of this size introduces real memory and computation constraints that need to be handled carefully.

2.2. Objectives

1. To study and understand content-based filtering, collaborative filtering, and hybrid recommendation approaches.
2. To implement a content-based recommendation module using song metadata and cosine similarity.
3. To implement an item-based collaborative filtering module using user listening history, with efficient sparse matrix storage.
4. To combine both modules into a single hybrid system using a weighted score approach.
5. To manage code and data versions using GitHub and DVC.

3. METHODOLOGY / APPROACH

3.1. Overall Workflow

The project is built around three stages: content-based filtering, collaborative filtering, and a hybrid combination of both. Figure 1, Figure 2, and Figure 3 illustrate each stage in detail.

The first stage takes raw song metadata through a cleaning and transformation pipeline to produce a feature matrix, which is then used to find similar songs via cosine similarity. The second stage

processes user listening history using Dask and scipy sparse matrices to build a compact interaction matrix for collaborative similarity computation. The third stage normalises the scores from both modules and merges them using a weighted sum to generate the final recommendations.

3.2. Technical Details

Content-Based Filtering

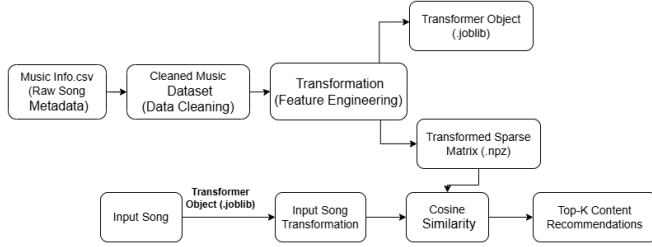


Fig. 1. Content-Based Filtering Pipeline

The first file in the dataset contains song metadata: artist names, genres, tags, and various audio descriptors. Before doing anything with it, I cleaned the data by dropping rows with missing values and converting all text to lowercase. After that, the text features were converted into numerical vectors using TF-IDF (Term Frequency-Inverse Document Frequency), a technique that assigns higher weight to words that are distinctive across the dataset and lower weight to words that appear everywhere.

Once every song was represented as a vector, I computed similarity scores using cosine similarity:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \quad (1)$$

where $A \cdot B$ is the dot product of the two song vectors, and $\|A\|$, $\|B\|$ are their respective magnitudes. Cosine similarity measures the angle between two vectors rather than their size, so a song with a longer description does not automatically score higher than a shorter one. The output always falls between -1 and $+1$, which makes scores from different songs directly comparable..

Collaborative Filtering

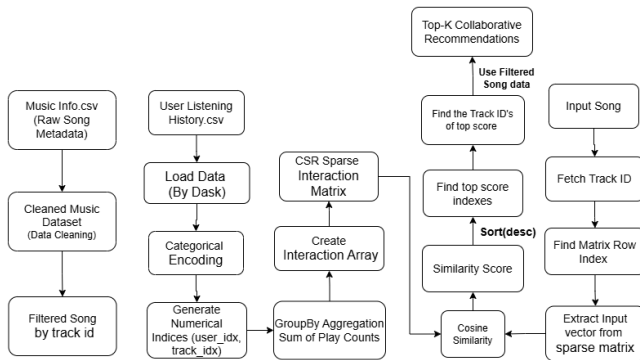


Fig. 2. Collaborative Filtering Pipeline

The second file records user listening history, specifically how many times each user played each song (the play count). I used **item-based collaborative filtering** for this module, where songs occupy the rows and users occupy the columns of the interaction matrix. Similarity is computed between songs rather than between users.

The reason for choosing item-based over user-based filtering is practical: the dataset has roughly 30,000 unique songs but around 9.7 million unique users. With songs in rows, the matrix has far fewer rows to work with, which makes similarity computations significantly faster and keeps memory usage under control.

Handling the Large Interaction Matrix

The biggest engineering challenge in this project was the size of the user-item matrix. Trying to load it as a standard pandas DataFrame would exhaust most machines because pandas loads everything into RAM at once. I tackled this with three techniques:

- **Dask for parallel computation:** Dask is a Python library whose syntax closely mirrors pandas and NumPy, so adapting my existing code was straightforward. Rather than loading the full dataset at once, Dask breaks it into chunks and processes them across CPU cores in parallel. It uses a *client* to receive commands, a *scheduler* to figure out what can run simultaneously, and *workers* to execute each chunk. A particularly useful property is that Dask is lazy by default: nothing actually runs until `.compute()` is called, which keeps memory usage low until it is genuinely needed.
- **CSR sparse matrix for storage:** Because most users have only heard a small fraction of all available songs, the interaction matrix is mostly zeros. A standard DataFrame allocates memory for every zero, which is extremely wasteful at this scale. Scipy's **CSR (Compressed Sparse Row)** format avoids this by storing only the non-zero values and their positions in the matrix. This reduced the matrix from what would have been hundreds of gigabytes down to just 31.5 MB.
- **Integer encoding of indices:** The song and user IDs in the raw data are long string values. Before constructing the matrix, I replaced each unique ID with an integer index. Integers consume much less memory than strings, and this alone noticeably reduced the size of the index arrays and sped up lookups during similarity computation.

Hybrid Combination

Once both modules produce their similarity scores, the next step is combining them. Before doing that, both sets of scores are normalised to the same scale using min-max normalisation. Without this step, the module that naturally returns larger values would dominate the final result regardless of actual recommendation quality.

The hybrid score is then computed as a weighted sum:

$$s_{\text{hybrid}} = w_1 \cdot s_{\text{content}} + w_2 \cdot s_{\text{collab}} \quad (2)$$

where:

- s_{hybrid} = final combined similarity score
- s_{content} = normalised score from the content-based module
- s_{collab} = normalised score from the collaborative module
- w_1, w_2 = weights for each module, where $w_1 + w_2 = 1$

The weights are user-controlled. A higher w_1 produces recommendations that closely resemble the input song, while a higher w_2 introduces more variety. The top 5 to 10 songs with the highest s_{hybrid} scores are returned as the final output.

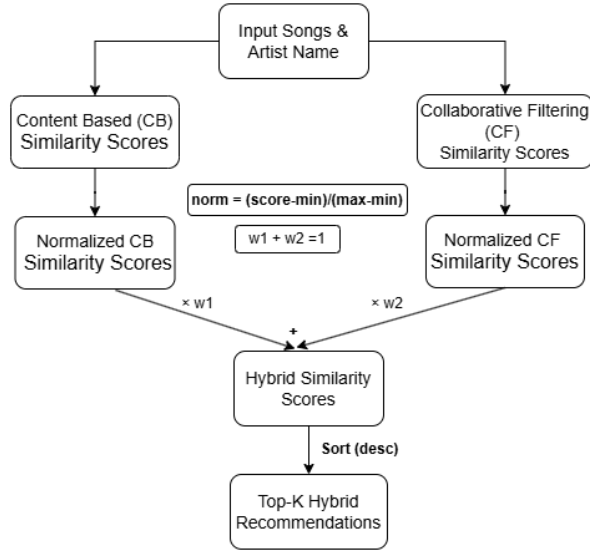


Fig. 3. Hybrid Recommender System Pipeline

4. EXPERIMENTS AND RESULTS

4.1. Experimental Setup

The dataset consists of two files. The first contains metadata for approximately 30,000 unique songs. The second holds user listening history with play counts across around 9.7 million unique users.

All code was written in Python using pandas and NumPy for general data handling, scipy for sparse matrix operations, Dask for large-scale computation, and scikit-learn for feature transformation and similarity calculations. Code versioning was handled through GitHub and data pipeline versioning through DVC, structured so that a single `dvc repro` command runs the full workflow from raw data to recommendations.

Since this is a recommendation task rather than a classification one, standard metrics like accuracy or F1 score do not apply here. No ground truth labels exist in the dataset, so evaluation was done qualitatively: I checked whether the returned songs made contextual sense for a given input, and I varied the weights w_1 and w_2 to observe how the output shifted between similarity and diversity.

4.2. Results

The content-based module consistently returned songs whose metadata was close to the input, with the most similar tracks appearing at the top of the list. The collaborative filtering module brought in songs that other users with overlapping listening habits had played, which added variety that the content-based results alone would not have included.

When the two modules were combined, adjusting the weights produced clearly different outputs. Higher w_1 values kept the recommendations stylistically close to the input. Higher w_2 values opened up the list to more unexpected but plausible suggestions. The fact that both directions behaved predictably confirmed that the hybrid logic was working correctly.

5. DISCUSSION

One of the biggest lessons from this project came from a mistake I made early on. I started building the user-item interaction matrix using standard pandas without thinking carefully about the scale. With over 960,000 users and 30,000 tracks, the matrix had billions of potential cells. When I tried to load it, my laptop froze completely. That was the moment I realised I was dealing with a data engineering problem, not just a modelling one.

That experience led me to Dask. The first thing that made it appealing was how closely its syntax matches pandas, so I did not have to rewrite my logic from scratch. I just adapted what I already had, and the difference was immediate. The computation ran where it had previously crashed. It was the first time I genuinely understood why tools built for scale exist and what parallel computing actually feels like when it works.

Dask alone did not fully solve the memory problem though. The matrix itself was the real issue. Since most users have only heard a small portion of all available songs, the vast majority of the matrix is zeros. A pandas DataFrame allocates memory for every one of those zeros. Switching to Scipy’s CSR format fixed this because it only stores the values that are actually non-zero, along with their positions. In practice, this brought the matrix size down from what would have been hundreds of gigabytes to just 31.5 MB, which made the whole collaborative module practical to run on a regular machine.

The speed improvement that came with it was something I did not fully anticipate. Because the matrix was now so compact, fetching a song’s row and comparing it against the rest of the data became near-instant. Recommendations that might otherwise have taken minutes were returning in seconds.

On the cold start side, the approach was more straightforward. A new user with no listening history receives popularity-based suggestions as a starting point. The moment they play a single song, the content-based module takes over and begins recommending similar tracks using that song’s metadata. It is not a perfect fix, but it means the system always has something reasonable to show, even on a user’s very first visit.

6. CONCLUSION AND FUTURE WORK

6.1. Conclusion

Going into this project, I assumed building a recommendation system was mostly about picking the right similarity measure. By the end, I realised that was only a small part of it. The harder problems were around data: how to store it efficiently, how to structure it for fast computation, and how to keep the pipeline from collapsing under its own scale.

One idea that genuinely shifted how I think about recommendations is that a useful system does not only reflect what one user likes. It draws patterns from the behaviour of many users and uses that to surface things the individual might never have discovered on their own. That is what makes collaborative filtering worth the engineering effort.

Handling a sparse matrix with nearly 9.7 million users also gave me a much clearer understanding of memory management than any coursework had. Dask, CSR format, and integer encoding were not optional enhancements. They were the minimum required to make the project runnable at all on consumer hardware.

I also came to genuinely appreciate pipeline-based code structure. Setting up DVC felt like unnecessary overhead at first. Once it was running, everything became easier to trace and re-run. Just

reading the pipeline made it clear which script produced which output and in what order, and that kind of transparency is something I will carry into future work. **Future Work**

There are several directions I want to take this project further, some of which I already have a rough plan for.

6.2. Future Work

There are several directions I want to take this project further.

1. **Smart Search and Autocomplete:** Currently, the app requires exact song and artist names. I want to add a dynamic search that suggests matches as the user types, and auto-fills the artist field based on the selected track. This would reduce invalid inputs and make the interface much easier to use.
2. **Evaluation Through User Feedback:** There are no ground truth labels in the dataset, so measuring recommendation quality objectively is not currently possible. Adding a simple thumbs up or thumbs down option specifically for recommendations would gradually build a labelled dataset that could support proper evaluation.
3. **Implicit Feedback Loop:** Explicit ratings are rarely given by real users. Passive signals like skipping a song within a few seconds or replaying it immediately say a great deal about preference without asking anything of the user. Feeding these signals back into the model over time would make recommendations progressively more accurate.
4. **Dynamic Weight Adjustment:** The weights w_1 and w_2 are currently set manually. I want to replace this with a lightweight model that reads a user's recent activity and predicts the weights automatically, so the balance between familiarity and discovery adjusts itself without any input from the user.
5. **API Deployment:** The system runs as a Streamlit app, which works for demonstration but is not easy to build on top of. Wrapping the recommendation logic in a REST API would allow other applications to use it directly and make the system easier to scale and maintain as the dataset grows.
6. **Language Expansion:** The current dataset is almost entirely English and has no regional music. I want to collect and integrate metadata for Hindi and other language tracks so the system can serve a wider audience. This matters to me personally because the users I most want this system to reach are not exclusively English-language listeners.

7. ARTIFACTS AND DEMONSTRATIONS

- **Code Repository:**
<https://github.com/SagarPundir368/Spotify-Hybrid-Recom>
Contains the complete source code, DVC pipelines, and data processing scripts.
- **Live Application:**
<https://spotify-hybrid-recom-nq2ejztctp5czqgx3xyne5.streamlit.app/>
An interactive web interface demonstrating the hybrid recommendation engine.
- **Video Demonstration:**
<https://youtu.be/u9JkXGwSVA4>

A brief video explaining the project architecture and a live test of the system.

8. REFERENCES

- [1] S. Peyton Jones, "How to write a great research paper," Microsoft Research, 2003.
- [2] Dask Development Team, "10 Minutes to Dask," *Dask Documentation*. [Online]. Available: <https://docs.dask.org/en/stable/10-minutes-to-dask.html>.
- [3] Kaggle, "Million Song Dataset with User Listening History," Available: <https://www.kaggle.com/datasets/undefinenu1/million-song-dataset-spotify-lastfm>